



Cairo University Racing Team

Formula Student / FSAE®

Generative AI Team

(Technical Questions)

Season 26-27

Table of Contents

PART 1 Questions	3
PART 2 Design an AI assistant for CURT	11
My thought process:	11
system architecture	13
first system: Document Ingestion	13
Second system: AI RAG Assistant	14
Third system: Evaluation system	16
b. Main Challenges in the System	17
Data preprocessing and handling different file formats	17
Inconsistent, incomplete, or noisy data	17
Chunking and token/context limits	18
Retrieval of relevant information	19
c. Suggest at least two techniques or solutions to overcome these challenges.	20
1. Robust document preprocessing + OCR.....	20
2. Metadata + document versioning.....	20
3. Intelligent chunking.....	21
4. Hybrid retrieval + reranking.....	22
d. Explain how you would evaluate whether the system is working accurately and reliably	23
How do I know all of this actually works?.....	23
Evaluate the retrieval.....	23
Evaluate the generated answer	23
Test different types of questions	23

PART 1 Questions

1. Explain the difference between generative and discriminative models. Give one example of each.

in a nutshell

generative: from the name it is used to generate data it learns from the distribution of the training data given to it and generates new data based on the data it trains on example ChatGPT (I have more details in the second question)

discriminative: used to classify or predict an outcome similar to logistic regression is an email spam or not is the machine faulted or not

2. What is the purpose of transformers in modern AI? How does it work briefly?

Before discussing transformers, I would like to say the history of the transformer architecture and how it evolves over time

Before transformers we used normal RNNs until google said let's add a new layer or technique and we will call it the attention mechanism it really helped the LLM into understanding the whole sentence better after that they decided to work around the attention mechanism and remove the RNN and that is the first modern transformer architecture after that OPENAI made a new paper and removed the encoder from the google design and called it GPT (generative pretrained transformer)

Now how does a transformer work? I will take the GPT design for my explanation as it's simpler and widely used in LLMs in a nutshell the transformer takes a sentence and outputs a sentence + new word and we feed the output to the input again and again until we make the full sentence but how does this work internally

The transformer has many layers mainly:

- positional encoding
- masked multi head attention (important layer)
- feed forward
- soft max

we get the embedding and start to represent them into multi dimension space (this form retains the position and the meaning of each word)

after that they pass through the masked multi-head attention to know which word is important to other words and it's masked as the model only sees the words before the

current word and can't see what is after (no cheating) we get the Q K V vectors needed and a lot of linear algebra then boom we move to the next layer

wait we made a lot of math that doesn't relate to the meaning or position and those info might fade so we add a residual link to retain the previous signal

then we pass the new vectors to a feed forward network then SoftMax for determining the probabilities

3. Describe the difference between supervised, unsupervised, and reinforcement learning.

That was the first question I answered in my pattern recognition course haha

Supervised: the model is given labeled data for each input I know the correct output and that kind of data is the most expensive as it requires humans in the cycle (I think now this claim is debatable)

Unsupervised: the model learns from unlabeled data and he will try to classify them into clusters together according to some features

Reinforcement: this is a method of making the model learn over time by giving rewards or penalties and recently while I was researching for a graduation project idea, I came across a very interesting way of RL that doesn't require human it's called RLCD:

REINFORCEMENT LEARNING FROM CONTRASTIVE DISTILLATION FOR LM ALIGNMENT
take a look at the paper I encourage that

4. What are tokens in the context of Large Language Models? Why are they important?

Tokens are the main thing that makes a machine understand a word in the first place and it's the step zero that I didn't mention in my transformer explanation so let's go into the history, shall we?

- 1) Let's make each word has a number
 - a) cat = 1 dog = 2 the model will maybe say wait dog = 2 cat that is wrong and numbers doesn't have a meaningful relationship
- 2) one hot encoding: each word will have a unique vector dog = [0,1] cat = [1,0]
 - a) but we don't just has 2 words in English we have A LOOOTT of words too much space is needed
- 3) Let's make each letter a token
 - a) we will lose the intent and the actual meanings of each word

- 4) Let's make each word a token
 - a) we will have walk, walking, walked, walks all as different tokens even though they are the same
- 5) so, let's be in the middle and choose 'walk' only and ing or ed or s or tion ...as separate that can be used with any other word and this will help the model to generalize better for example: the model knows communicate but we gave it communication it will be able to understand it even though it never saw it

Ahh, that was tiring in a nutshell:

Computers don't naturally understand words like humans do A neural network works with numbers, so we need a way to convert language into numerical representations.

5. Explain the concept of attention in neural networks. How does it improve model performance?

I explained the history of the attention in the transformer layer, but I will get into more detail here

So, the attention mechanism allows a neural network to dynamically determine which parts of input most relevant to the current word or prediction and the model will assign different weights to different words based on their relationship

And the famous example: the bank of the river
is it a bank for money or bank of a river

the attention will make the model distinguish between them more accurately according to the context

in the transformer question I said Q K V vectors, but what are they?

- **Query (Q):** What information am I looking for?
- **Key (K):** What information does each token contain that can be matched against the query?
- **Value (V):** What information should be retrieved from that token?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

QK^T operation calculates how strongly tokens relate to each other and SoftMax for the weights scores

How they improve the model?

- 1) Focus on relevant information instead of treating each input equally

- 2) Process sequence more efficiently parallel (using the multi-head masked attention layers) not one at a time like RNN
- 3) Capture long range dependency better than RNNs

6. What is prompt engineering, and why is it crucial for Gen AI applications? State the different types of prompts used.

Before answering I will talk about problems I personally faced or seen from friends during my work for the What'sNew app I wrote into my cv I wanted to use AI for text enrichment and I want it to output a JSON file to store into firestore but the AI sometimes doesn't follow the structure given so that will require a block to sanitize the output and making sure that it follows my structure and re prompt it again making me waste more money and time

And that's where prompt engineering comes better prompt better output you will get with AI and LLMs you must be very specific with it so the problem with my prompt that it was zero shot with no examples so let's talk about prompts types

1) **Zero-shot prompting**

Give the model a task **without providing examples**.

2) **One-shot prompting**

Give **one example** before asking the model to perform the task.

3) **Few-shot prompting**

Give **multiple examples** to demonstrate the desired behavior.

4) **Role-based prompting**

Assign the model a specific **role or perspective**.

5) **Instructional prompting**

Give explicit **instructions and constraints** about what the model should do.

6) **Structured prompting**

Specify the desired **format or structure** of the output, such as JSON, a table, or a particular schema.

In my project I used a mixture of **Structured prompting** and **One-shot prompting**

Fun fact: recently this problem is solved by a new model called JEV where it only output according to the specified schema and it's very very cheap too

7. Explain how RAG (Retrieval-Augmented Generation) works.

It's a process where the LLM retrieve relevant information from an external knowledge source to build a more context aware output

User Query → Retrieval → Relevant Documents → Add to Prompt → LLM → Answer

So let's say I want to ask the LLM about how many quizzes do I have the LLM doesn't know but it can connect to my calendar or faculty website so here is the process

- 1) User ask question
- 2) Covert to embedding
- 3) Retrieve relevant info
- 4) Combine the prompt with the new Retrieved info

(Typically this example will use an MCP to make)

And using RAG will make the model be able to use your up to date data without retraining the model or producing wrong hallucinated answer

8. What is embedding in AI, and why is it important for retrieval tasks?

An embedding is a numerical vector representation of data any data (text, images, audio..) so similar or related items are near to each other king and queen or boy and girl (note we will notice that king and boy are away from each other with the same distance queen and girl are using cosine similarity which why embeddings are important)

What about RAG how are they connected

Suppose you have thousands of documents:

Documents → Embedding Model → Vector representations → Vector Database

The user query:

User Query → Embedding Model → Query Vector

Then we compare the query vector with the document vectors using cosine similarity then we get the most relevant vector

Also why not just search? Because embedding also carry the semantic meaning (I added a note on how to improve this in the **Explain the possible improvements that can be applied to a Naive RAG** question)

9. How does temperature affect the output of a language generation model?

Temperature is the meter that adds some randomness to the model response

Low: more predictable and deterministic

high: probability distribution becomes flatter and the results are more diverse

10. What is chain-of-thought prompting, and why can it improve reasoning in LLMs?

It's a technique used for making the LLM think step by step to go the final answer it's helpful to break complex problems into sequence of minor problems and it can improve performance

also, this question reminded me of the paper stealing reasoning traces where Chinese models were stealing the chain of thought to train their own models using distillation was an interesting paper very actually

11. What is the difference between RAG and fine-tuning? Give a use case for each.

Like I hinted in the Explain how RAG (Retrieval-Augmented Generation) works. RAG isn't finetuning here are the differences

RAG (Retrieval-Augmented Generation) provides the LLM with external information at inference time. The model itself is not retrained relevant documents are retrieved from a knowledge base and added to the model's context before generating the answer.

Use case: I am asking about quizzes I have next week or what policies for the IT registration

Fine-tuning involves training an already-pretrained model on a specific dataset so that its parameters are adjusted to perform a particular task or follow a particular style or behavior.

Use case: I have an image recognition model, but I want to finetune it to distinguish between different plants I want it to specialize in plants only

12. Explain the possible improvements that can be applied to a Naive RAG (Retrieval-Augmented Generation) system, both pre-retrieval and post-retrieval.

pre-retrieval:

- 1) we can separate the documents into different sized chunks rather than fixed ones like
 - Semantic chunking
 - Paragraph chunking
- 2) Query rewriting

The user: How do I fix login?

better query: What are the documented solutions for authentication and login failures?

3) Query expansion

The user: GPU memory problem

more terms related:

- GPU memory problem
- CUDA out of memory
- VRAM allocation failure
- GPU memory overflow

4) Combine keyword search and semantic search

5) Better embedding model

post-retrieval:

- 1) Reranking the given vectors instead of giving the top k blindly
- 2) Remove irrelevant documents
- 3) Compact or compress them if they are very long
- 4) Context reordering

13. What factors should you consider when choosing an LLM for an application?

Consider accuracy, latency, cost, context window, privacy, and reasoning ability.

I don't really understand this question as the answer is in the question, but I will elaborate on that

How will the model perform for my task and bigger doesn't mean better nor expensive means better

what if I will run it on a phone bigger model won't fit in the memory!

what if I process millions of text more expensive will make me go bankrupt!

does the model supports Arabic?

do I really need reasoning ?

do I need it to have tool calling capabilities?

is the data private and can't be shared? Will I use a local model?

is it being fast important to me? How important?

We must answer those questions according to the task we want and see the tradeoffs and take the most optimal for our case

14. How can you evaluate whether an LLM-generated answer is grounded in the provided documents?

Compare between the answer and the documents so we must source check

We can also use another LLM as a judge to compare automatically

Also, weird fact I noticed with AI - it's funny - that if they made up a fact and you ask it again "did you make that up?" it will answer so innocently "Yes you caught me"

15. How do you think computer vision could be integrated with Generative AI ?

Computer vision allows AI to understand visuals and generative AI generate about that content they can be combined to create a multimodal

Here are some applications

1) Image understanding

What is happening here in the image? then the model explains in natural language

2) Image → Text

Can add caption, descriptions, reports or summaries

X-ray → visual analysis → generated medical report

Also was used in YouTube the most recent feature auto dub it takes audio from the video and re translate it using the same audio from the video

3) Nano banana and image editing

Using generative AI and image generation we can upload an image and request an edit to be done using this multi-model approach

4) Video understanding and generation

Computer vision can extract information from video, while an LLM can reason over it and generate a summary

Also I recently saw a very interesting paper and a model for video generation from a prompt without the model hallucinate using a new thing in the middle called proxy where it generates a mini world with more accurate XYZ coordinates and more data then feed it to the video generation model

Here is the paper: <https://www.alphaxiv.org/pdf/2608.25927>

PART 2 Design an AI assistant for CURT

My thought process:

Before jumping into the system design let's analyze the requirements

Members ask questions about documents that must be a **RAG** system as those documents are very changeable and can't be in the training phase okay, great that is the first keyword Now different teams can access the LLM well that is a problem I don't want Mechanical team to see documents from Electrical team or Software or Business each team must see their related documents and from my journey as a software engineer I can't trust the user to do that task we will need an **authentication** mechanism but how will I know the document is related to the team or not hmm I will need to define metadata for each document

We will need **another separate system** just for the data how will I parse it and how will I add metadata to it then embeddings then store it in a vector database

Why metadata? I don't want to just store the document I want to add more context to it like

- Team
- Document name
- Pages
- Section
- Who can access it? (this might be useful if there is some kind of document that are related to different teams or an Admin role)

So, the metadata will solve

- **Authentication**
- **Citation**
- **Filtering**

There is a hint in the task that the model can access structured data if necessary well that is important as sometimes I will need to search for a specific gadget or tool they have distinct name already I think I might add a hybrid mechanism to handle this and a re-rank system that will decide what to take but wait this doesn't has anything to do with sql that is a normal search so we will have hybrid **normal search and vector search**

The database is for different reasons, like how many projects we have now but that will require a **function calling mechanism for the LLM we don't want an LLM to have direct access**

How to evaluate failure? We will have two kinds of failures to take care of

1. **Retrieval failure:** The correct document exists, but the system retrieves the wrong document
2. **Generation failure:** the correct document was retrieved but the LLM response is wrong.

So, when evaluating we will answer those two questions

1. Did we retrieve the right information?
2. Given the retrieved information, did the LLM answer correctly?

After researching I found that the first question for the RAG system has metrics like

- Recall at K: measures the share of relevant items captured within the top K positions.
- Precision at K: measures how many items with the top K positions are relevant.
- MRR: measures how early the first relevant result appears.

So, there will be three questions

- Out of all the relevant documents/chunks that exist, how many did I retrieve in my top K results?
- Of the K things I retrieved, how many were relevant?
- How high was the first relevant result?

What about generation failure

The answer must be supported by the retrieved information and must be correct

And that is called **Groundedness** I don't want the AI to add more info that is not supported by any document even if the answer seems correct

Is the answer relevant or the LLM just answer another question

If the answer is correct, does it include all the needed information or does the LLM missed info?

In a nutshell this task will need three systems in that order

- 1) Document Ingestion
- 2) AI RAG Assistant
- 3) Evaluation System

a. system architecture

a) here is the whiteboard link:

https://webwhiteboard.com/board/uXjVHkrs3Ho=?boardAccessToken=YnTJYncWOZL8zU7A4glmQEm5j9yKZYM_u

first system: Document Ingestion

Handles everything on the DOCUMENTS side

Its job is to turn raw documents into searchable knowledge.

1) Documents:

The system may receive different types of files:

- PDF
- DOCX
- CSV
- TXT
- Images/scanned documents

The first challenge is that these formats cannot all be directly processed by the LLM or embedding model.

2) Parse + OCR + Clean:

- a) Parsing extracts text from documents.
- b) OCR is needed for info inside an image rather than text.
- c) Cleaning removes unnecessary artifacts such as broken formatting, duplicated text, or irrelevant characters.

The goal is to produce clean information that can be processed consistently.

3) Chunk + Metadata:

A large document should not be stored as one giant piece

Mechanical_Design.pdf

Chunk 1 → Introduction

Chunk 2 → Engine specifications

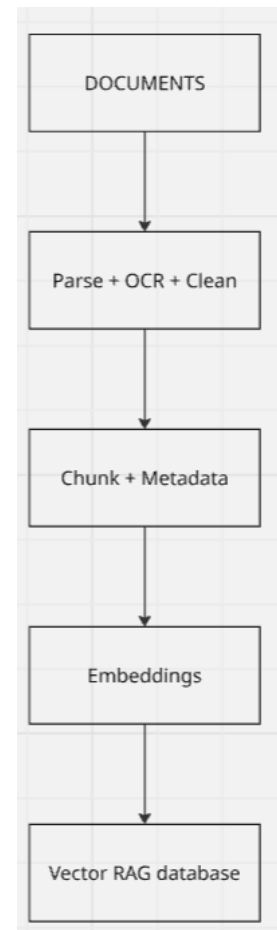
Each chunk also receives metadata.

This is extremely useful later because the retrieval system can use the metadata to enforce permissions, filter results, and generate citations

4) Embeddings:

Each chunk is converted into an embedding The purpose is to allow the system to find information based on **meaning**, rather than requiring an exact keyword match

5) Vector RAG Database:

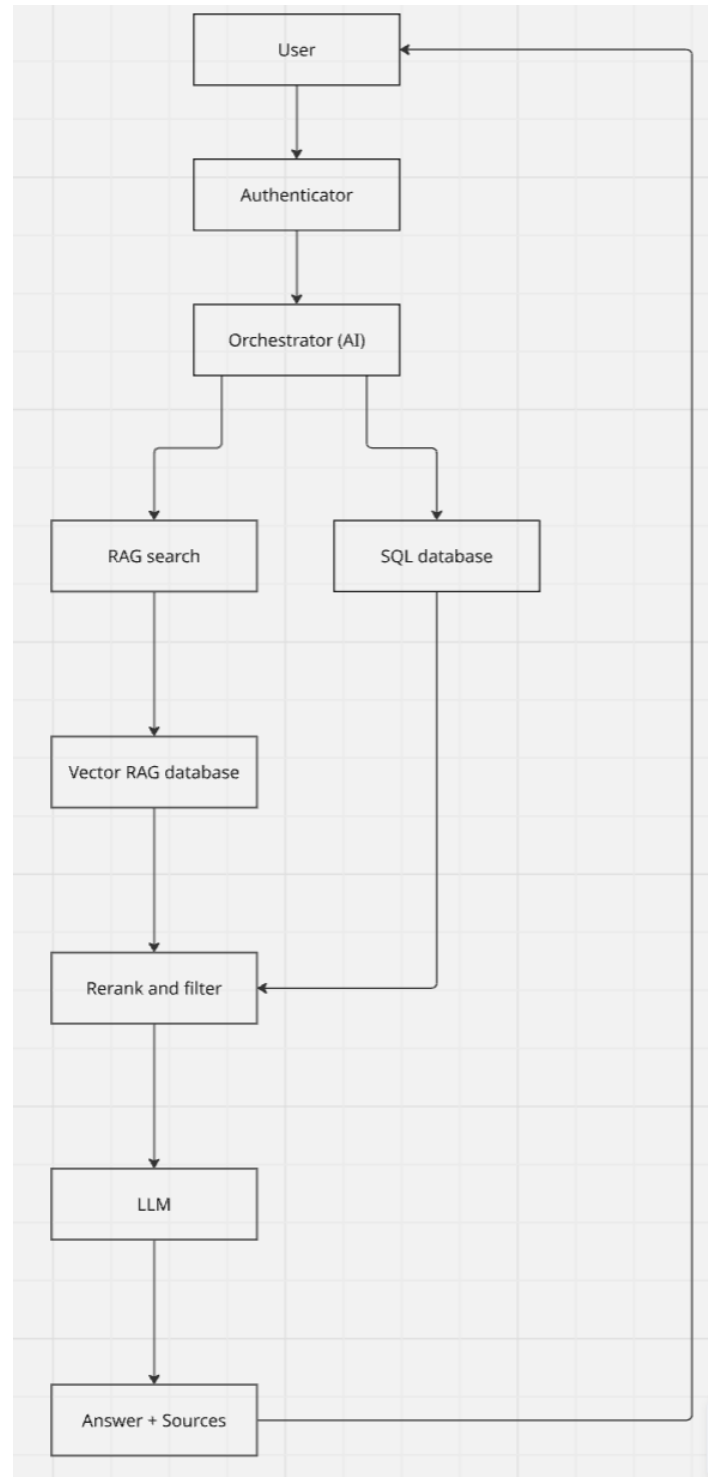


The embeddings and their associated metadata are stored in a **vector** database.

Second system: AI RAG Assistant

The second system handles actual interaction with the user

- 1) User: will query the assistant
- 2) Authenticator: The first step is to identify the user
we need to know the user identity and what it can access
The user's permissions can then be compared against the access metadata attached to chunks
this is done before LLM step so the LLM can't see unauthorized data
- 3) AI Orchestrator: this is our controller will determine how the question shall be handles structured data or RAG system?
- 4) RAG Search:
 - a) If the user asks something about the organization's documents, the orchestrator sends the query to the RAG system. However, I would not rely exclusively on vector search
 - b) Hybrid Search: there are two different types of searches that might be useful
 - i) Semantic search
when user express an idea that is not necessarily like the documents
 - ii) Keyword/normal search
useful when the query contains code or identifiers that the user will know of like a mechanical tool well known
- 5) Rerank + Filter: The initial retrieval stage may return several candidate chunks.
They aren't necessarily all equally useful.



A reranker evaluates the candidates and determines which ones are most relevant to the specific query!

- 6) SQL Database: the info not necessarily should be in a document for example:

How many active projects do we currently have?

Searching a PDF for this information isn't necessarily correct

The LLM can use a **function/tool calling mechanism** to request the correct database operation.

I would **not give the LLM unrestricted direct access to the database**.

Instead, the LLM should interact through controlled tools/functions

- 7) LLM: Once the system has collected the necessary information the LLM will generate the final result

the instruction should also tell the model to:

- i) answer using the provided evidence
- ii) avoid unsupported claims
- iii) state when the information isn't available
- iv) provide citations

Third system: Evaluation system

How do we know whether our RAG assistant actually works?

A RAG system can fail in two major places.

Retrieval Failure

The correct information exists in the database, but the retrieval system doesn't retrieve it.

We can evaluate using

- Recall at K: measures the share of relevant items captured within the top K positions.
- Precision at K: measures how many items with the top K positions are relevant.
- MRR: measures how early the first relevant result appears.

Generation Failure

Now imagine the retrieval worked perfectly
Now the question will be:
Given the retrieved information, did the LLM produce a good answer?

Correctness

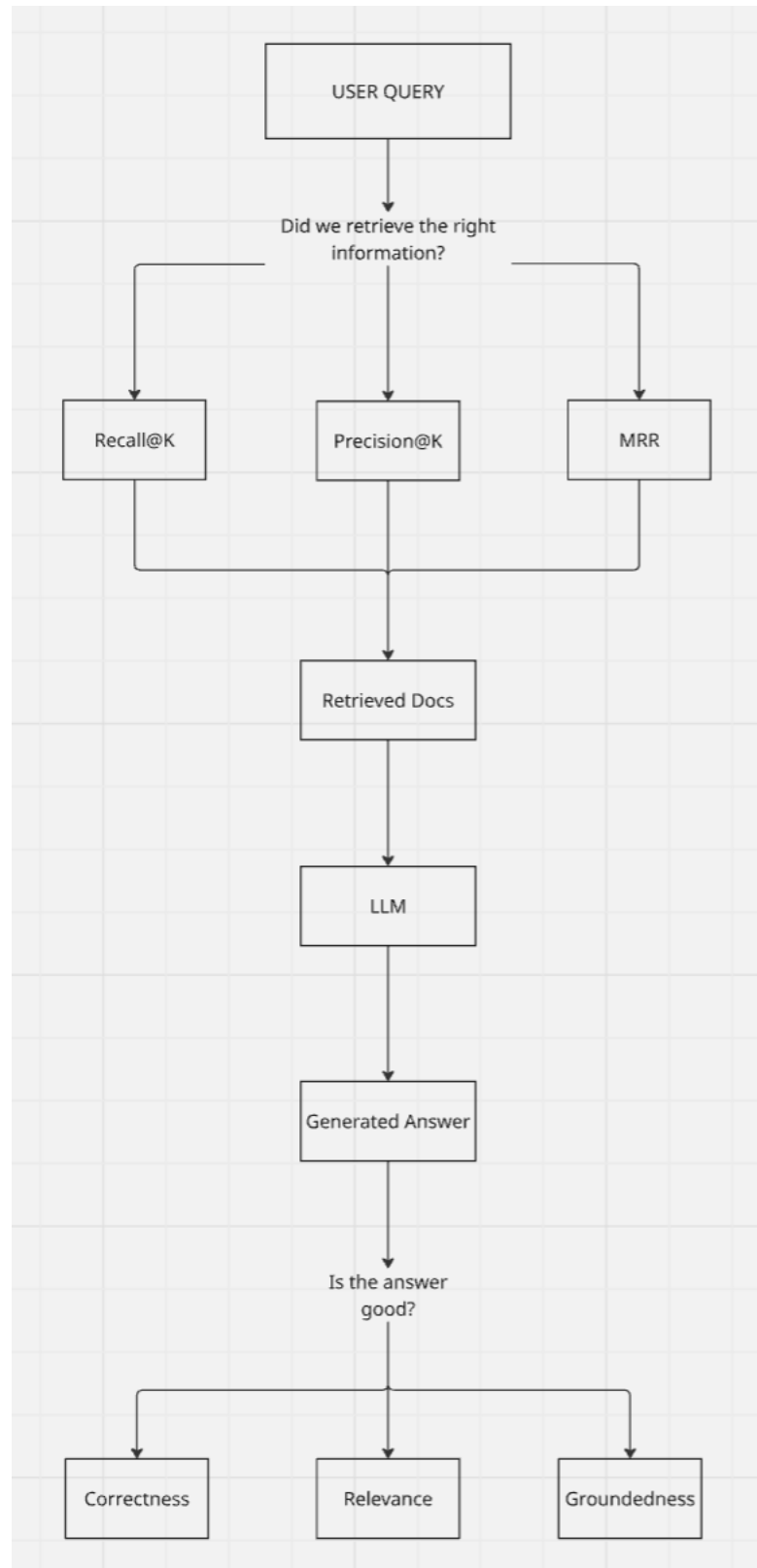
Is the answer factually correct?

Groundedness

Is the answer supported by the retrieved information?

Relevance

Does the answer actually answer the user's question?



b. Main Challenges in the System

Data preprocessing and handling different file formats

The organization can have many types of documents:

- PDF
- DOCX
- PPTX
- XLSX
- CSV
- TXT
- Scanned documents/images

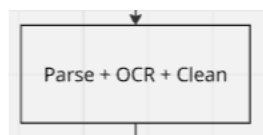
Each format stores information differently. For example, a PDF may contain normal text or scanned pages, while an Excel file contains structured tables.

the challenge is:

How can we convert all these different formats into a consistent representation that the RAG system can understand?

extracting text may not work because the information is actually stored as an image.

This is why our **Document Ingestion System** contains:



Inconsistent, incomplete, or noisy data

Even after extracting the data, the documents themselves may contain problems.

- Duplicate information
- Missing values
- Broken formatting
- Typos
- Conflicting versions
- Irrelevant text
- Incorrect information

If all of them are indexed without handling the conflict, the retriever may return contradictory information to the LLM

This creates two problems:

- **Retrieval problems:** irrelevant or outdated information may be retrieved.
- **Generation problems:** the LLM may use incorrect or conflicting information to produce an answer.

Therefore, data quality needs to be handled **before and during indexing**, rather than expecting the LLM to clean up everything.

Chunking and token/context limits

Large documents cannot simply be sent to the LLM as one huge piece of text.

Sending the entire document to the LLM would:

- consume a huge number of tokens
- increase computational cost
- exceed the model's context limit in some cases
- provide the model with a lot of irrelevant information

Therefore, the document needs to be divided into **smaller chunks**.

But chunking creates another challenge.

If the chunks are:

Too large Lots of unrelated information Retrieval becomes less precise and wastes context.

Too small

- Chunk 1 → "The maximum pressure is"
- Chunk 2 → "250 bar under normal..."

Important information may be split between chunks.

So the challenge is:

How do we choose chunk sizes and boundaries that preserve enough context while keeping retrieval precise and the LLM's context manageable?

Retrieval of relevant information

Semantic vs exact search

A user might ask: What pressure can the hydraulic system handle?

while the document says: Maximum operating pressure: 250 bar.

A vector search can recognize the semantic similarity.

But if the user asks: What does component HYD-PRV-204 do?

an exact search may be more useful.

Hybrid search is my solution

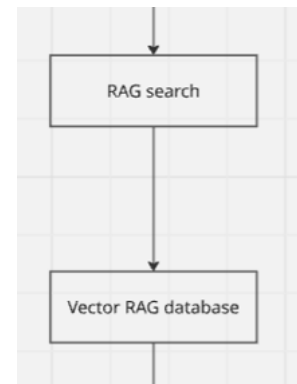
Accuracy of generated answers

Even if retrieval is perfect, the LLM can still produce an incorrect answer.

This is a **generation failure**.

Another possibility is that the LLM adds information that wasn't present in the retrieved documents:

Can the LLM generate an answer that is correct, relevant, complete, and supported by the retrieved information?



c. Suggest at least two techniques or solutions to overcome these challenges.

Now that we identified the problems, let's talk about how I would actually solve them.

Some of the solutions were already mentioned while explaining the challenges, but here I will put them together and explain how they would be implemented in the system.

1. Robust document preprocessing + OCR

The first problem is that our documents can come in many different formats, so I can't just give every file directly to the embedding model.

I would make the **Document Ingestion System** responsible for converting all of them into a common representation like I mentioned before.

For normal documents we can use the appropriate parser to extract the text.

For scanned documents or images, we use **OCR** to convert the text inside the image into actual text.

Then we clean the extracted information by removing things like:

- duplicate text
- broken formatting
- unnecessary characters
- extraction artifacts

This gives us a consistent representation before we start chunking and generating embeddings.

2. Metadata + document versioning

I already mentioned metadata when explaining the architecture, but this is actually one of the most important solutions because it solves more than one problem.

Instead of storing only the chunk itself, I would store something like:

- Team: Mechanical
- Document: Hydraulic_Design.pdf

- Version: 3
- Page: 15
- Section: Pressure Requirements
- Access: Mechanical, Admin
- Date: 2026

Now we have much more information about the chunk.

For example, if a Mechanical user asks a question, the retrieval system can filter the results based on the user's permissions.

It can also help us deal with different versions of the same document.

If we have:

- Hydraulic_Design_v1
- Hydraulic_Design_v2
- Hydraulic_Design_v3

we don't want the system to randomly retrieve an old version when a newer valid version exists.

And there is another benefit that I mentioned earlier: **citations**.

Because we stored the document name, page, and section, the system can tell the user exactly where the answer came from.

So one metadata system helps us with:

- **Authorization**
- **Filtering**
- **Version control**
- **Citations**

3. Intelligent chunking

For the chunking problem, I wouldn't just take a document and split it every 500 characters or something like that.

I would try to understand the structure of the document first. The idea is to keep related information together.

I can also use a small overlap between chunks so that important information isn't suddenly lost at the boundary.

We don't want chunks so large that retrieval becomes noisy and wastes the LLM's context, but we also don't want them so small that we destroy the meaning of the original document.

4. Hybrid retrieval + reranking

I already mentioned this while discussing the retrieval problem.

I don't think using only vector search is enough for this system.

So I would combine:

1. Vector / Semantic Search
2. Keyword / Exact Search

and then use a **reranker**.

d. Explain how you would evaluate whether the system is working accurately and reliably

Now we have another problem:

How do I know all of this actually works?

A system can fail in two major places, which I mentioned earlier:

1. **Retrieval failure**
2. **Generation failure**

So I would evaluate them separately.

Evaluate the retrieval

First, I want to answer:

Did we retrieve the right information?

I can use three metrics:

- **Recall@K** How many of the relevant documents/chunks did I manage to retrieve within my
- **Precision@K** Out of the K things that I retrieved, how many were actually relevant?
- **MRR** How high was the first relevant result?

Evaluate the generated answer

- **Correctness** Is the answer actually correct?
- **Groundedness** Is the answer supported by the retrieved documents?
- **Relevance** Did the LLM actually answer the question?
- **Completeness** Did the answer contain all the important information needed to answer the question?

Test different types of questions

Finally, I wouldn't evaluate the system using only simple questions.

I would create different types of test cases:

- Normal questions
- Questions requiring multiple documents
- Questions with spelling mistakes

- Ambiguous questions
- Questions where the answer doesn't exist
- Questions involving conflicting document versions
- Questions requiring SQL/structured data
- Questions containing exact component IDs
- Unauthorized questions